

1 Intro to basics

R makes use of the # sign to add comments.

In [1]:
```
# This is a comment

#install.packages("devtools")
IRkernel::installspec()
```

It is possible to make calculations without call any library

In [2]:
```
6 + 18 + 5
```

29

1.1 Arithmetic with R

In its most basic form, R can be used as a simple calculator. Consider the following arithmetic operators:

- Addition: +
- Subtraction: -
- Multiplication: *
- Division: /
- Exponentiation: ^
- Modulo: %%

The ^ operator raises the number to its left to the power of the number to its right: for example 3^2 is 9.

The modulo returns the remainder of the division of the number to the left by the number on its right, for example 5 modulo 3 or 5 %% 3 is 2.

In [4]:
```
3^3
```

27

In [5]:
```
11 %% 5
```

1

1.2 Variable assignment

A basic concept in (statistical) programming is called a variable.

A variable allows you to store a value (e.g. 4) or an object (e.g. a function description) in R. You can then later use this variable's name to easily access the value or the object that is stored within this variable.

You can assign a value 4 to a variable my_var with the command

In [1]:
```
my_var <- 4
```

Now, let to print the result

In [2]:
```
my_var
```

4

In [3]:
```
my_var <- 9
```

In [4]:
```
my_var
```

9

1.2.1 Variable assignment (2)

Suppose you have a fruit basket with five apples. As a data analyst in training, you want to store the number of apples in a variable with the name my_apples.

In [5]:
```
my_apples <- 5
```

In [6]:
```
my_apples
```

5

1.2.2 Variable assignment (3)

Every tasty fruit basket needs pears, so you decide to add six pears. As a data analyst, your reflex is to immediately create the variable my_pears and assign the value 6 to it. Next, you want to calculate how many pieces of fruit you have in total. Since you have given meaningful names to these values, you can now code this in a clear way:

In [12]:
```
my_apples + my_pears
```

```
Error in eval(expr, envir, enclos): object 'my_pears' not found
Traceback:
```

In [13]:
```
my_pears <- 6
```

In [14]:
```
my_apples + my_pears
```

11

In [15]:
```
my_fruit <- my_apples + my_pears
```

In [16]:
```
my_fruit
```

11

Common knowledge tells you not to add apples and pears. But hey, that is what you just did, no :-)? The my_apples and my_pears variables both contained a number in the previous exercise. The + operator works with numeric variables in R. If you really tried to add "apples" and "pears", and assigned a text value to the variable my_pears, you would be trying to assign the addition of a numeric and a character variable to the variable my_fruit. This is not possible.

In [19]:
```
my_pears <- "six"
```

In [20]:
```
my_fruit <- my_apples + my_pears
```

```
Error in my_apples + my_pears: non-numeric argument to binary operator
Traceback:
```

1.3 Basic data types in R

R works with numerous data types. Some of the most basic types to get started are:

- Decimal values like 4.5 are called numerics.
- Whole numbers like 4 are called integers. Integers are also numerics.
- Boolean values (TRUE or FALSE) are called logical.
- Text (or string) values are called characters.

Note how the quotation marks in the editor indicate that "some text" is a string.

In [21]:
```
# Change my_numeric to be -4512
my_numeric <- 42

# Change my_character to be "multiverse"
my_character <- "universe"

# Change my_logical to be TRUE
my_logical <- FALSE
```

What's that data type?

Do you remember that when you added 5 + "six", you got an error due to a mismatch in data types? You can avoid such embarrassing situations by checking the data type of a variable beforehand. You can do this with the class() function, as the code in the editor shows.

In [22]:
```
# Check class of my_numeric
class(my_numeric)

# Check class of my_character
class(my_character)

# Check class of my_logical
class(my_logical)
```

'numeric'

'character'

'logical'

1.3.1 Vectors

1.3.1.1 Create a vector

Now, you on a trip to the City of Sins, also known as Statisticians Paradise!

Thanks to R and your new data-analytical skills, you will learn how to uplift your performance at the tables and fire off your career as a professional gambler. This chapter will show how you can easily keep track of your betting progress and how you can do some simple analyses on past actions.

On your way from rags to riches, you will make extensive use of vectors. Vectors are one-dimension arrays that can hold numeric data, character data, or logical data. In other words, a vector is a simple tool to store data. For example, you can store your daily gains and losses in the casinos.

In R, you create a vector with the combine function **c()**. You place the vector elements separated by a comma between the parentheses. For example:

In [24]:
```
numeric_vector <- c(1, 10, 49)
character_vector <- c("a", "b", "c")

numeric_vector

character_vector
```

1 10 49

'a' 'b' 'c'

Once you have created these vectors in R, you can use them to do calculations.

Complete the code such that **boolean_vector** contains the three elements: TRUE, FALSE and TRUE (in that order).

In [28]:
```
boolean_vector <- c(T, F, T)

boolean_vector
```

TRUE FALSE TRUE

After one week in Las Vegas and still zero Ferraris in your garage, you decide that it is time to start using your statistical superpowers.

Before doing a first analysis, you decide to first collect all the winnings and losses for the last

week:

For poker_vector:

- On Monday you won $140
- Tuesday you lost $50
- Wednesday you won $20
- Thursday you lost $120
- Friday you won $240

For roulette_vector:

- On Monday you lost $24
- Tuesday you lost $50
- Weesday you won $100
- Thursday you lost $350
- Friday you won $10

In [25]:
```
# Poker winnings from Monday to Friday
poker_vector <- c(140, -50, 20, -120, 240)

poker_vector
```

140 -50 20 -120 240

In [26]:
```
# Roulette winnings from Monday to Friday
roulette_vector <-  c(-24, -50, 100, -350, 10)

roulette_vector
```

-24 -50 100 -350 10

As a data analyst, it is important to have a clear view on the data that you are using. Understanding what each element refers to is therefore essential.

1.3.1.2 Naming a vector

In the previous exercise, we created a vector with your winnings over the week. Each vector element refers to a day of the week but it is hard to tell which element belongs to which day. It would be nice if you could show that in the vector itself.

You can give a name to the elements of a vector with the **names()** function. Have a look at this example:

In [28]:
```
# Assign days as names of poker_vector
names(poker_vector) <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")
```

In [29]:
```
poker_vector
```

Monday	140
Tuesday	-50
Wednesday	20
Thursday	-120
Friday	240

In the previous exercises you probably experienced that it is boring and frustrating to type and retype information such as the days of the week. However, when you look at it from a higher perspective, there is a more efficient way to do this, namely, to assign the days of the week vector to a variable!

In [30]:
```
# The variable days_vector
days_vector <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")
```

In [31]:
```
# Assign the names of the day to roulette_vector and poker_vector
names(poker_vector) <-   days_vector
names(roulette_vector) <- days_vector
```

In [32]:
```
poker_vector

roulette_vector
```

Monday	140
Tuesday	-50
Wednesday	20
Thursday	-120
Friday	240

Monday	-24
Tuesday	-50
Wednesday	100
Thursday	-350
Friday	10

1.3.1.3 Adding vectors

Lets calculate some properties

- How much has been your overall profit or loss per day of the week?
- Have you lost money over the week in total?
- Are you winning/losing money on poker or on roulette?

To get the answers, you have to do arithmetic calculations on vectors.

It is important to know that if you sum two vectors in R, it takes the element-wise sum. For example, the following three statements are completely equivalent:

In [34]:
```
c(1, 8, 3) + c(4, 5, 6)
c(1 + 4, 8 + 5, 3 + 6)
c(5, 13, 9)
```

```
5  13  9
```

```
5  13  9
```

```
5  13  9
```

You can also do the calculations with variables that represent vectors:

In [35]:
```
a <- c(1, 8, 3)
b <- c(4, 5, 6)
 c <- a + b
```

In [36]:
```
c
```

```
5  13  9
```

How much has been your overall profit or loss per day of the week?

In [37]:
```
# Poker and roulette winnings from Monday to Friday:
poker_vector <- c(140, -50, 20, -120, 240)
roulette_vector <- c(-24, -50, 100, -350, 10)
days_vector <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")
names(poker_vector) <- days_vector
names(roulette_vector) <- days_vector
```

In [38]:
```
# Assign to total_daily how much you won/lost on each day
total_daily <- poker_vector + roulette_vector
```

In [39]:
```
total_daily
```

Monday	116
Tuesday	-100
Wednesday	120
Thursday	-470
Friday	250

Based on the previous analysis, it looks like you had a mix of good and bad days. This is not what your ego expected, and you wonder if there may be a very tiny chance you have lost money over the week in total?

A function that helps you to answer this question is **sum()**. It calculates the sum of all elements of a vector. For example, to calculate the total amount of money you have lost/won with poker you do:

In [40]:
```
# Total winnings with poker
total_poker <- sum(poker_vector)
total_poker
```

230

In [42]:
```
# Total winnings with roulette
total_roulette <- sum(roulette_vector)
total_roulette
```

-314

In [43]:
```
# Total winnings overall
total_week <- total_poker + total_roulette

# Print out total_week
  total_week
```

-84

Oops, it seems like you are losing money. Time to rethink and adapt your strategy! This will require some deeper analysis…

you realize that a possible explanation might be that your skills in roulette are not as well developed as your skills in poker. So maybe your total gains in poker are higher (or >) than in roulette.

In [44]:
```
# Calculate total gains for poker and roulette
total_poker <- sum(poker_vector)
total_roulette <- sum(roulette_vector)

# Check if you realized higher total gains in poker than in roulette
total_poker > total_roulette
```

TRUE

1.3.1.4 Vector selections

Your hunch seemed to be right. It appears that the poker game is more your cup of tea than roulette.

Another possible route for investigation is your performance at the beginning of the working week compared to the end of it.

To answer that question, you only want to focus on a selection of the total_vector. In other words, our goal is to select specific elements of the vector. To select elements of a vector (and later matrices, data frames, …), you can use square brackets.

Between the square brackets, you indicate what elements to select. For example, to select the first element of the vector, you type **poker_vector[1]**. To select the second element of the vector, you type **poker_vector[2]**, etc. Notice that the first element in a vector has **index 1, not 0** as in many other programming languages.

In [45]:
```
# Poker and roulette winnings from Monday to Friday:
poker_vector <- c(140, -50, 20, -120, 240)
roulette_vector <- c(-24, -50, 100, -350, 10)
days_vector <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")
names(poker_vector) <- days_vector
names(roulette_vector) <- days_vector
```

In [46]:
```
# Define a new variable based on a selection
poker_wednesday <- poker_vector[3]
```

In [47]:
```
poker_wednesday
```

Wednesday: 20

To select multiple elements from a vector, you can add square brackets at the end of it. You can indicate between the brackets what elements should be selected. For example: suppose you want to select the first and the fifth day of the week: use the vector c(1, 5) between the square brackets. For example, the code below selects the first and fifth element of poker_vector:

In [48]:
```
poker_vector[c(1, 5)]
```

Monday	140
Friday	240

In [49]:
```
# Define a new variable based on a selection
poker_midweek <- poker_vector[c(2,3,4)]
```

In [50]:
```
poker_midweek
```

Tuesday	-50
Wednesday	20
Thursday	-120

Another way

In [51]:
```
poker_midweek <- poker_vector[c(2:4)]
```

In [52]:
```
poker_midweek
```

Tuesday	-50
Wednesday	20
Thursday	-120

Another way to tackle the previous exercise is by using the names of the vector elements (Monday, Tuesday, …) instead of their numeric positions. For example,

In [54]:
```
poker_vector["Monday"]
```

Monday: 140

In [57]:
```
# Select poker results for Monday, Tuesday and Wednesday
poker_start <- poker_vector[c("Monday", "Tuesday", "Wednesday")]


poker_start

# Calculate the average of the elements in poker_start
mean(poker_start)
```

Monday	140
Tuesday	-50
Wednesday	20

36.6666666666667

Selection by comparison

By making use of comparison operators, we can approach the previous question in a more proactive way.

The (logical) comparison operators known to R are:

- < for less than
- > for greater than
- <= for less than or equal to
- >= for greater than or equal to
- == for equal to each other
- != not equal to each other

In [58]:
```
c(4, 5, 6) > 5
```

FALSE FALSE TRUE

This command tests for every element of the vector if the condition stated by the comparison operator is TRUE or FALSE.

In [59]:
```
# Which days did you make money on poker?
selection_vector <- poker_vector > 0

# Print out selection_vector
selection_vector
```

Monday	TRUE
Tuesday	FALSE
Wednesday	TRUE
Thursday	FALSE
Friday	TRUE

Working with comparisons will make your data analytical life easier. Instead of selecting a subset of days to investigate yourself (like before), you can simply ask R to return only those days where you realized a positive return for poker.

In the previous exercises you used **selection_vector <- poker_vector > 0** to find the days on which you had a positive poker return. Now, you would like to know not only the days on which you won, but also how much you won on those days.

You can select the desired elements, by putting selection_vector between the square brackets that follow poker_vector:

In [60]:
```
poker_vector[selection_vector]
```

Monday	140
Wednesday	20
Friday	240

R knows what to do when you pass a logical vector in square brackets: it will only select the elements that correspond to TRUE in selection_vector.

1.3.2 Matrix

In R, a matrix is a collection of elements of the same data type (numeric, character, or logical) arranged into a fixed number of rows and columns. Since you are only working with rows and columns, a matrix is called two-dimensional.

You can construct a matrix in R with the **matrix()** function. Consider the following example:

In [2]:
```
matrix(1:9, byrow = T, nrow = 3)
```

```
 1  2  3
 4  5  6
 7  8  9
```

In [4]:
```
matrix(1:9, byrow = F, nrow = 2)
```

```
Warning message in matrix(1:9, byrow = F, nrow = 2):
"data length [9] is not a sub-multiple or multiple of the number of rows [2]"
```

```
 1  3  5  7  9
 2  4  6  8  1
```

In the matrix() function:

- The first argument is the collection of elements that R will arrange into the rows and columns of the matrix. Here, we use 1:9 which is a shortcut for c(1, 2, 3, 4, 5, 6, 7, 8, 9).
- The argument byrow indicates that the matrix is filled by the rows. If we want the matrix to be filled by the columns, we just place byrow = FALSE.
- The third argument nrow indicates that the matrix should have three rows.

In the following exercises you will analyze the box office numbers of the Star Wars franchise.

In [7]:
```
# Box office Star Wars (in millions!)
new_hope <- c(460.998, 314.4)
empire_strikes <- c(290.475, 247.900)
return_jedi <- c(309.306, 165.8)
```

Three vectors are defined. Each one represents the box office numbers from the first three Star Wars movies. The first element of each vector indicates the US box office revenue, the second element refers to the Non-US box office.

to combine the three vectors into one vector

In [8]:
```
# Create box_office
box_office <- c(new_hope, empire_strikes, return_jedi)
```

In [9]:
```
box_office
```

```
460.998  314.4  290.475  247.9  309.306  165.8
```

Construct a matrix, where each row represents a movie (ordered by rows). The first argument is the vector box_office.

In [10]:
```
# Construct star_wars_matrix
star_wars_matrix <- matrix(box_office, nrow = 3, byrow = T)
star_wars_matrix
```

460.998	314.4
290.475	247.9
309.306	165.8

1.3.2.1 Naming a matrix

To help you remember what is stored in star_wars_matrix, you would like to add the names of the movies for the rows. Not only does this help you to read the data, but it is also useful to select certain elements from the matrix.

Similar to vectors, you can add names for the rows and the columns of a matrix:

rownames(my_matrix) <- row_names_vector

colnames(my_matrix) <- col_names_vector

In [2]:
```
# Box office Star Wars (in millions!)
new_hope <- c(460.998, 314.4)
empire_strikes <- c(290.475, 247.900)
return_jedi <- c(309.306, 165.8)

# Construct matrix
star_wars_matrix <- matrix(c(new_hope, empire_strikes, return_jedi), nrow = 3, by

star_wars_matrix

# Vectors region and titles, used for naming
region <- c("US", "non-US")
titles <- c("A New Hope", "The Empire Strikes Back", "Return of the Jedi")

# Name the columns with region
rownames(star_wars_matrix) <- titles

# Name the rows with titles
colnames(star_wars_matrix) <- region

# Print out star_wars_matrix
star_wars_matrix
```

460.998	314.4
290.475	247.9
309.306	165.8

	US	non-US
A New Hope	460.998	314.4
The Empire Strikes Back	290.475	247.9
Return of the Jedi	309.306	165.8

To calculate the total box office revenue for the three Star Wars movies, you have to take the sum of the US revenue column and the non-US revenue column.

In R, the function **rowSums()** conveniently calculates the totals for each row of a matrix. This function creates a new vector:

In [14]:
```
rowSums(star_wars_matrix)
```

A New Hope	775.398
The Empire Strikes B…	538.375
Return of the Jedi	475.106

rowSums() has a **colSums()** to add columns

In [15]:
```
colSums(star_wars_matrix)
```

US	1060.779
non-US	728.1

In the previous exercise you calculated the vector that contained the worldwide box office receipt for each of the three Star Wars movies. However, this vector is not yet part of star_wars_matrix.

You can add a column or multiple columns to a matrix with the **cbind()** function, which merges matrices and/or vectors together by column. For example:

In [3]:
```
# Construct star_wars_matrix
box_office <- c(460.998, 314.4, 290.475, 247.900, 309.306, 165.8)
region <- c("US", "non-US")
titles <- c("A New Hope",
            "The Empire Strikes Back",
            "Return of the Jedi")

star_wars_matrix <- matrix(box_office,
                      nrow = 3, byrow = TRUE,
                      dimnames = list(titles, region))

# The worldwide box office figures
worldwide_vector <- rowSums(star_wars_matrix)
income_source <- colSums(star_wars_matrix)

star_wars_matrix
worldwide_vector
income_source

# Bind the new variable worldwide_vector as a column to star_wars_matrix
all_wars_matrix <- cbind(star_wars_matrix, worldwide_vector)

all_wars_matrix
```

	US	non-US
A New Hope	460.998	314.4
The Empire Strikes Back	290.475	247.9
Return of the Jedi	309.306	165.8

A New Hope	775.398
The Empire Strikes B…	538.375
Return of the Jedi	475.106

US	1060.779
non-US	728.1

	US	non-US	worldwide_vector
A New Hope	460.998	314.4	775.398
The Empire Strikes Back	290.475	247.9	538.375
Return of the Jedi	309.306	165.8	475.106

In [4]:
```
all_wars_matrix <- cbind(star_wars_matrix, worldwide_vector, income_source)

all_wars_matrix
```

```
Warning message in cbind(star_wars_matrix, worldwide_vector, income_source):
"number of rows of result is not a multiple of vector length (arg 3)"
```

	US	non-US	worldwide_vector	income_source
A New Hope	460.998	314.4	775.398	1060.779
The Empire Strikes Back	290.475	247.9	538.375	728.100
Return of the Jedi	309.306	165.8	475.106	1060.779

Just like every action has a reaction, every **cbind** has an **rbind**

In [5]:
```
all_wars_matrix2 <- rbind(star_wars_matrix, income_source)

all_wars_matrix2
```

	US	non-US
A New Hope	460.998	314.4
The Empire Strikes Back	290.475	247.9
Return of the Jedi	309.306	165.8
income_source	1060.779	728.1

Similar to vectors, you can use the square brackets [ ] to select one or multiple elements from a matrix. Whereas vectors have one dimension, matrices have two dimensions. You should therefore use a comma to separate the rows you want to select from the columns. For example:

my_matrix[1,2] selects the element at the first row and second column.

my_matrix[1:3,2:4] results in a matrix with the data on the rows 1, 2, 3 and columns 2, 3, 4.

If you want to select all elements of a row or a column, no number is needed before or after the comma, respectively:

my_matrix[,1] selects all elements of the first column.

my_matrix[1,] selects all elements of the first row.

In [7]:
```
# Box office Star Wars (in millions!)
new_hope <- c(460.998, 314.4)
empire_strikes <- c(290.475, 247.900)
return_jedi <- c(309.306, 165.8)
The_Phantom_Menace <- c(474.5, 552.5)
Attack_of_the_Clones <-   c(310.7, 338.7)
Revenge_of_the_Sith   <-  c(380.3, 468.5)

# Construct matrix
all_wars_matrix <- matrix(c(new_hope, empire_strikes, return_jedi, The_Phantom_Me
                            nrow = 6,
                            byrow = TRUE)

# Vectors region and titles, used for naming
region <- c("US", "non-US")
titles <- c("A New Hope", "The Empire Strikes Back", "Return of the Jedi", "The P

# Name the columns with region
rownames(all_wars_matrix) <- titles

# Name the rows with titles
colnames(all_wars_matrix) <- region

all_wars_matrix
```

	US	non-US
A New Hope	460.998	314.4
The Empire Strikes Back	290.475	247.9
Return of the Jedi	309.306	165.8
The Phantom Menace	474.500	552.5
Attack of the Clones	310.700	338.7
Revenge of the Sith	380.300	468.5

Select the non-US revenue for all movies (the entire second column of all_wars_matrix), store the result as non_us_all.

In [9]:
```
non_us_all <- all_wars_matrix[,1]
non_us_all
```

A New Hope	460.998
The Empire Strikes B…	290.475
Return of the Jedi	309.306
The Phantom Menace	474.5
Attack of the Clones	310.7
Revenge of the Sith	380.3

Calculate the average non-US revenue

In [28]:
```
f <- all_wars_matrix[,2]
mean(f)
```

347.966666666667

Select the non-US revenue for first two movies

In [25]:
```
non_us_some <- all_wars_matrix[1:2, 2]
non_us_some
```

A New Hope	314.4
The Empire Strikes B…	247.9

1.3.2.2 A little arithmetic with matrices

Similar to what you have learned with vectors, the standard operators like +, -, /, *, etc. work in an element-wise way on matrices in R.

For example, 2 * my_matrix multiplies each element of my_matrix by two.

As a newly-hired data analyst for Lucasfilm, it is your job to find out how many visitors went to each movie for each geographical area. You already have the total revenue figures in **all_wars_matrix**. Assume that the price of a ticket was 5 dollars. Simply dividing the box office numbers by this ticket price gives you the number of visitors.

Divide all_wars_matrix by 5, giving you the number of visitors in millions. Assign the resulting matrix to visitors

In [10]:
```
# Estimate the visitors
visitors <- all_wars_matrix/5

# Print the estimate to the console
visitors
```

	US	non-US
A New Hope	92.1996	62.88
The Empire Strikes Back	58.0950	49.58
Return of the Jedi	61.8612	33.16
The Phantom Menace	94.9000	110.50
Attack of the Clones	62.1400	67.74
Revenge of the Sith	76.0600	93.70

Just like 2 * my_matrix multiplied every element of my_matrix by two, **my_matrix1 * my_matrix2** creates a matrix where each element is the **product** of the corresponding elements in my_matrix1 and my_matrix2.

1.3.3 Factors

The term factor refers to a statistical data type used to store categorical variables. The difference between a categorical variable and a continuous variable is that a categorical variable can belong to a limited number of categories. A continuous variable, on the other hand, can correspond to an infinite number of values.

It is important that R knows whether it is dealing with a continuous or a categorical variable, as the statistical models you will develop in the future treat both types differently.

A good example of a categorical variable is sex. In many circumstances you can limit the sex categories to "Male" or "Female". (Sometimes you may need different categories. For example, you may need to consider chromosomal variation, hermaphroditic animals, or different cultural norms, but you will always have a finite number of categories.)

To create factors in R, you make use of the function **factor()**. First thing that you have to do is create a vector that contains all the observations that belong to a limited number of categories. For example, sex_vector contains the sex of 5 different individuals:

In [31]:
```
sex_vector <- c("Male","Female","Female","Male","Male")
sex_vector
```

'Male' 'Female' 'Female' 'Male' 'Male'

It is clear that there are two categories, or in R-terms 'factor levels', at work here: "Male" and "Female".

The function factor() will encode the vector as a factor:

In [32]:
```
factor_sex_vector <- factor(sex_vector)
factor_sex_vector
```

Male Female Female Male Male

▶ **Levels**:

There are two types of **categorical variables: a nominal categorical variable and an ordinal categorical variable**.

A **nominal variable** is a categorical variable without an implied order. This means that it is impossible to say that 'one is worth more than the other'. For example, think of the categorical variable animals_vector with the categories "Elephant", "Giraffe", "Donkey" and "Horse". Here, it is impossible to say that one stands above or below the other.

In contrast, **ordinal variables** do have a natural ordering. Consider for example the categorical variable temperature_vector with the categories: "Low", "Medium" and "High". Here it is obvious that "Medium" stands above "Low", and "High" stands above "Medium".

In [11]:
```
# Animals
animals_vector <- c("Elephant", "Giraffe", "Donkey", "Horse")
factor_animals_vector <- factor(animals_vector)
factor_animals_vector

# Temperature
temperature_vector <- c("High", "Low", "High","Low", "Medium")
factor_temperature_vector <- factor(temperature_vector, order = TRUE, levels = c(
factor_temperature_vector
```

Elephant Giraffe Donkey Horse

▶ **Levels**:

High Low High Low Medium

▶ **Levels**:

When you first get a dataset, you will often notice that it contains factors with specific factor levels. However, sometimes you will want to change the names of these levels for clarity or other reasons. R allows you to do this with the function **levels()**

levels(factor_vector) <- c("name1", "name2",...)

A good illustration is the raw data that is provided to you by a survey. A common question for every questionnaire is the sex of the respondent. Here, for simplicity, just two categories were recorded, "M" and "F".

In [63]:
```
survey_vector <- c("M", "F", "F", "M", "M")
```

Recording the sex with the abbreviations "M" and "F" can be convenient if you are collecting data with pen and paper, but it can introduce confusion when analyzing the data. At that point, you will often want to change the factor levels to "Male" and "Female" instead of "M" and "F" for clarity.

Watch out: the order with which you assign the levels is important. If you type levels(factor_survey_vector), you'll see that it outputs [1] "F" "M". If you don't specify the levels of the factor when creating the vector, R will automatically assign them alphabetically. To correctly map "F" to "Female" and "M" to "Male", the levels should be set to c("Female", "Male"), in this order.

In [12]:
```
# Code to build factor_survey_vector
survey_vector <- c("M", "F", "F", "M", "M")
factor_survey_vector <- factor(survey_vector)
factor_survey_vector

# Specify the levels of factor_survey_vector
levels(factor_survey_vector) <- c("Female", "Male")

factor_survey_vector
```

M F F M M

▶ **Levels**:

Male Female Female Male Male

▶ **Levels**:

1.3.3.1 Summarizing a factor

After finishing this course, one of your favorite functions in R will be **summary()**. This will give you a quick overview of the contents of a variable:

summary(my_var)

In [13]:
```
# Build factor_survey_vector with clean levels
survey_vector <- c("M", "F", "F", "M", "M")
factor_survey_vector <- factor(survey_vector)
levels(factor_survey_vector) <- c("Female", "Male")


# Generate summary for survey_vector
summary(survey_vector)

# Generate summary for factor_survey_vector
summary(factor_survey_vector)
```

```
   Length     Class      Mode 
        5 character character 
```

Female	2
Male	3

Since "Male" and "Female" are unordered (or nominal) factor levels, R returns a warning message, telling you that the greater than operator is not meaningful. As seen before, R attaches an equal value to the levels for such factors.

But this is not always the case! Sometimes you will also deal with factors that do have a natural ordering between its categories. If this is the case, we have to make sure that we pass this information to R…

Let us say that you are leading a research team of five data analysts and that you want to evaluate their performance. To do this, you track their speed, evaluate each analyst as "slow", "medium" or "fast", and save the results in speed_vector.

As a first step, assign speed_vector a vector with 5 entries, one for each analyst. Each entry should be either "slow", "medium", or "fast". Use the list below:

- Analyst 1 is medium,
- Analyst 2 is slow,
- Analyst 3 is slow,
- Analyst 4 is medium and
- Analyst 5 is fast.

No need to specify these are factors yet.

In [ ]:
```
# Create speed_vector
speed_vector <- c("medium", "slow", "slow", "medium", "fast")
```

speed_vector should be converted to an **ordinal factor** since its categories have a natural ordering. By default, the function factor() transforms speed_vector into an unordered factor. To create an ordered factor, you have to add two additional arguments: **ordered** and **levels**.

factor(some_vector, ordered = TRUE, levels = c("lev1", "lev2" ...))

In [14]:
```
# Create speed_vector
speed_vector <- c("medium", "slow", "slow", "medium", "fast")

# Convert speed_vector to ordered factor vector
factor_speed_vector <- factor(speed_vector, ordered = T, levels = c("slow", "medi

# Print factor_speed_vector
factor_speed_vector
summary(factor_speed_vector)
```

medium slow slow medium fast

▶ **Levels**:

slow	2
medium	2
fast	1

1.3.3.2 Comparing ordered factors

Having a bad day at work, 'data analyst number two' enters your office and starts complaining that 'data analyst number five' is slowing down the entire project. Since you know that 'data analyst number two' has the reputation of being a smarty-pants, you first decide to check if his statement is true.

The fact that factor_speed_vector is now ordered enables us to compare different elements (the data analysts in this case). You can simply do this by using the well-known operators.

In [15]:
```
# Create factor_speed_vector
speed_vector <- c("medium", "slow", "slow", "medium", "fast")
factor_speed_vector <- factor(speed_vector, ordered = TRUE, levels = c("slow", "m

# Factor value for second data analyst
da2 <- factor_speed_vector[2]

# Factor value for fifth data analyst
da5 <- factor_speed_vector[5]

# Is data analyst 2 faster than data analyst 5?
da2 > da5
```

FALSE

1.3.4 Data frames

You may remember from the chapter about matrices that all the elements that you put in a matrix should be of the same type. Back then, your dataset on Star Wars only contained numeric

elements.

When doing a market research survey, however, you often have questions such as:

- 'Are you married?' or 'yes/no' questions (logical)
- 'How old are you?' (numeric)
- 'What is your opinion on this product?' or other 'open-ended' questions (character)
- …

The output, namely the respondents' answers to the questions formulated above, is a dataset of different data types. You will often find yourself working with datasets that contain different data types instead of only one.

A data frame has the variables of a dataset as columns and the observations as rows. This will be a familiar concept for those coming from different statistical software packages such as SAS or SPSS even Excel or csv files.

In [16]:
```
# Print out built-in R data frame
mtcars

#https://rstudio-pubs-static.s3.amazonaws.com/61800_faea93548c6b49cc91cd0c5ef5059
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

Working with large datasets is not uncommon in data analysis. When you work with (extremely) large datasets and data frames, your first task as a data analyst is to develop a clear understanding of its structure and main elements. Therefore, it is often useful to show only a small part of the entire dataset.

So how to do this in R? Well, the function **head()** enables you to show the first observations of a data frame. Similarly, the function **tail()** prints out the last observations in your dataset.

Both head() and tail() print a top line called the 'header', which contains the names of the different variables in your dataset.

In [39]:
```
# Call head() on mtcars
head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

Another method that is often used to get a rapid overview of your data is the function **str()**. The function str() shows you the structure of your dataset. For a data frame it tells you:

- The total number of observations (e.g. 32 car types)
- The total number of variables (e.g. 11 car features)
- A full list of the variables names (e.g. mpg, cyl …)
- The data type of each variable (e.g. num)
- The first observations

Applying the **str()** function will often be the first thing that you do when receiving a new dataset or data frame. It is a great way to get more insight in your dataset before diving into the real analysis.

In [23]:
```
# Investigate the structure of mtcars
str(mtcars)
```

```
'data.frame':   32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
 $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

In [24]:
```
summary(mtcars)
```

```
      mpg             cyl             disp             hp       
 Min.   :10.40   Min.   :4.000   Min.   : 71.1   Min.   : 52.0  
 1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5  
 Median :19.20   Median :6.000   Median :196.3   Median :123.0  
 Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7  
 3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0  
 Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0  
      drat             wt             qsec             vs        
 Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000  
 1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000  
 Median :3.695   Median :3.325   Median :17.71   Median :0.0000  
 Mean   :3.597   Mean   :3.217   Mean   :17.85   Mean   :0.4375  
 3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000  
 Max.   :4.930   Max.   :5.424   Max.   :22.90   Max.   :1.0000  
       am              gear            carb      
 Min.   :0.0000   Min.   :3.000   Min.   :1.000  
 1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000  
 Median :0.0000   Median :4.000   Median :2.000  
 Mean   :0.4062   Mean   :3.688   Mean   :2.812  
 3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000  
 Max.   :1.0000   Max.   :5.000   Max.   :8.000  
```

1.3.4.1 Creating a data frame

As a first goal, you want to construct a data frame that describes the main characteristics of eight planets in our solar system. The main features of a planet are:

- The type of planet (Terrestrial or Gas Giant).
- The planet's diameter relative to the diameter of the Earth.
- The planet's rotation across the sun relative to that of the Earth.
- If the planet has rings or not (TRUE or FALSE).

After doing some high-quality research on Wikipedia, you feel confident enough to create the necessary vectors: name, type, diameter, rotation and rings.

You construct a data frame with the data.frame() function. As arguments, you pass the vectors from before: they will become the different columns of your data frame. Because every column has the same length, the vectors you pass should also have the same length. But don't forget that it is possible (and likely) that they contain different types of data.

In [17]:
```
# Definition of vectors
name <- c("Mercury", "Venus", "Earth",
          "Mars", "Jupiter", "Saturn",
          "Uranus", "Neptune")

type <- c("Terrestrial planet",
          "Terrestrial planet",
          "Terrestrial planet",
          "Terrestrial planet", "Gas giant",
          "Gas giant", "Gas giant", "Gas giant")

diameter <- c(0.382, 0.949, 1, 0.532,
              11.209, 9.449, 4.007, 3.883)

rotation <- c(58.64, -243.02, 1, 1.03,
              0.41, 0.43, -0.72, 0.67)

rings <- c(FALSE, FALSE, FALSE, FALSE, TRUE, TRUE, TRUE, TRUE)

# Create a data frame from the vectors
planets_df <- data.frame(name, type, diameter, rotation, rings)
planets_df
```

name	type	diameter	rotation	rings
Mercury	Terrestrial planet	0.382	58.64	FALSE
Venus	Terrestrial planet	0.949	-243.02	FALSE
Earth	Terrestrial planet	1.000	1.00	FALSE
Mars	Terrestrial planet	0.532	1.03	FALSE
Jupiter	Gas giant	11.209	0.41	TRUE
Saturn	Gas giant	9.449	0.43	TRUE
Uranus	Gas giant	4.007	-0.72	TRUE
Neptune	Gas giant	3.883	0.67	TRUE

In [41]:
```
head(planets_df)
```

name	type	diameter	rotation	rings
Mercury	Terrestrial planet	0.382	58.64	FALSE
Venus	Terrestrial planet	0.949	-243.02	FALSE
Earth	Terrestrial planet	1.000	1.00	FALSE
Mars	Terrestrial planet	0.532	1.03	FALSE
Jupiter	Gas giant	11.209	0.41	TRUE
Saturn	Gas giant	9.449	0.43	TRUE

The planets_df data frame should have 8 observations and 5 variables.

In [26]:
```
# Check the structure of planets_df
str(planets_df)
```

```
'data.frame':   8 obs. of  5 variables:
 $ name    : Factor w/ 8 levels "Earth","Jupiter",..: 4 8 1 3 2 6 7 5
 $ type    : Factor w/ 2 levels "Gas giant","Terrestrial planet": 2 2 2 2 1 1 1
1
 $ diameter: num  0.382 0.949 1 0.532 11.209 ...
 $ rotation: num  58.64 -243.02 1 1.03 0.41 ...
 $ rings   : logi  FALSE FALSE FALSE FALSE TRUE TRUE ...
```

In [42]:
```
summary(planets_df)
```

```
     name                   type      diameter          rotation       
 Earth  :1   Gas giant         :4   Min.   : 0.3820   Min.   :-243.0200  
 Jupiter:1   Terrestrial planet:4   1st Qu.: 0.8448   1st Qu.:   0.1275  
 Mars   :1                          Median : 2.4415   Median :   0.5500  
 Mercury:1                          Mean   : 3.9264   Mean   : -22.6950  
 Neptune:1                          3rd Qu.: 5.3675   3rd Qu.:   1.0075  
 Saturn :1                          Max.   :11.2090   Max.   :  58.6400  
 (Other):2                                                               
  rings          
 Mode :logical  
 FALSE:4        
 TRUE :4        
```

1.3.4.2 Selection of data frame elements

Similar to vectors and matrices, you select elements from a data frame with the help of square brackets [ ]. By using a comma, you can indicate what to select from the rows and the columns respectively. For example:

- my_df[1,2] selects the value at the first row and second column in my_df.
- my_df[1:3,2:4] selects rows 1, 2, 3 and columns 2, 3, 4 in my_df.

Sometimes you want to select all elements of a row or column.

- For example, my_df[1,] selects all elements of the first row.

In [24]:
```
# The planets_df data frame from the previous exercise is pre-loaded
planets_df


# Print out data for Mars (entire fourth row)
planets_df[4,]

# Print out diameter of Mercury (row 1, column 3)
planets_df[3,][3:3]
```

name	type	diameter	rotation	rings
Mercury	Terrestrial planet	0.382	58.64	FALSE
Venus	Terrestrial planet	0.949	-243.02	FALSE
Earth	Terrestrial planet	1.000	1.00	FALSE
Mars	Terrestrial planet	0.532	1.03	FALSE
Jupiter	Gas giant	11.209	0.41	TRUE
Saturn	Gas giant	9.449	0.43	TRUE
Uranus	Gas giant	4.007	-0.72	TRUE
Neptune	Gas giant	3.883	0.67	TRUE

	name	type	diameter	rotation	rings
4	Mars	Terrestrial planet	0.532	1.03	FALSE

	diameter
3	1

Instead of using numerics to select elements of a data frame, you can also use the variable names to select columns of a data frame.

Suppose you want to select the first three elements of the type column. One way to do this is

In [25]:
```
planets_df[1:3,2]
```

Terrestrial planet Terrestrial planet Terrestrial planet

▶ **Levels**:

A possible disadvantage of this approach is that you have to know (or look up) the column number

of type, which gets hard if you have a lot of variables. It is often easier to just make use of the variable name:

In [51]:
```
planets_df[1:3,"rings"]
```

FALSE FALSE FALSE

In [26]:
```
# The planets_df data frame from the previous exercise is pre-loaded
planets_df

# Select first 5 values of diameter column
planets_df[1:5,"diameter"]
```

name	type	diameter	rotation	rings
Mercury	Terrestrial planet	0.382	58.64	FALSE
Venus	Terrestrial planet	0.949	-243.02	FALSE
Earth	Terrestrial planet	1.000	1.00	FALSE
Mars	Terrestrial planet	0.532	1.03	FALSE
Jupiter	Gas giant	11.209	0.41	TRUE
Saturn	Gas giant	9.449	0.43	TRUE
Uranus	Gas giant	4.007	-0.72	TRUE
Neptune	Gas giant	3.883	0.67	TRUE

0.382 0.949 1 0.532 11.209

You will often want to select an entire column, namely one specific variable from a data frame. If you want to select all elements of the variable diameter, for example, both of these will do the trick:

In [27]:
```
planets_df[,3]
```

0.382 0.949 1 0.532 11.209 9.449 4.007 3.883

In [28]:
```
planets_df[,"diameter"]
```

0.382 0.949 1 0.532 11.209 9.449 4.007 3.883

However, there is a short-cut. If your columns have names, you can use the $ sign:

In [30]:
```
planets_df$diameter
```

0.382 0.949 1 0.532 11.209 9.449 4.007 3.883

You probably remember from high school that some planets in our solar system have rings and others do not. Unfortunately you can not recall their names. Could R help you out?

In [58]:
```
rings_vector <- planets_df$rings

rings_vector
```

FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE

This means that the first four observations (or planets) do not have a ring (FALSE), but the other four do (TRUE).

Let's try to use rings_vector to select the data for the four planets with rings.

In [60]:
```
planets_df[rings_vector, ]
```

	name	type	diameter	rotation	rings
5	Jupiter	Gas giant	11.209	0.41	TRUE
6	Saturn	Gas giant	9.449	0.43	TRUE
7	Uranus	Gas giant	4.007	-0.72	TRUE
8	Neptune	Gas giant	3.883	0.67	TRUE

So what exactly did you learn in the previous exercises? You selected a subset from a data frame (planets_df) based on whether or not a certain condition was true (rings or no rings).

Now, let us move up one level and use the function **subset()**. You should see the subset() function as a short-cut to do exactly the same as what you did in the previous exercises.

subset(my_df, subset = some_condition)

The first argument of subset() specifies the dataset for which you want a subset. By adding the second argument, you give R the necessary information and conditions to select the correct subset.

The code below will give the exact same result as you got in the previous exercise, but this time, you didn't need the rings_vector

In [41]:
```
subset(planets_df, subset = rings)
```

	name	type	diameter	rotation	rings
5	Jupiter	Gas giant	11.209	0.41	TRUE
6	Saturn	Gas giant	9.449	0.43	TRUE
7	Uranus	Gas giant	4.007	-0.72	TRUE
8	Neptune	Gas giant	3.883	0.67	TRUE

In [42]:
```
# Select planets with diameter < 1
subset(planets_df, subset = diameter < 1)
```

	name	type	diameter	rotation	rings
1	Mercury	Terrestrial planet	0.382	58.64	FALSE
2	Venus	Terrestrial planet	0.949	-243.02	FALSE
4	Mars	Terrestrial planet	0.532	1.03	FALSE

1.3.4.3 Sorting

In data analysis you can sort your data according to a certain variable in the dataset. In R, this is done with the help of the function **order()**.

order() is a function that gives you the ranked position of each element when it is applied on a variable, such as a vector for example:

In [32]:
```
a <- c(100, 10, 1000)
order(a)
```

2 1 3

10, which is the second element in a, is the smallest element, so 2 comes first in the output of order(a). 100, which is the first element in a is the second smallest element, so 1 comes second in the output of order(a).

This means we can use the output of order(a) to reshuffle a:

In [33]:
```
a[order(a)]

?sqrt
```

10 100 1000

Alright, now that you understand the order() function, let us do something useful with it. You would like to rearrange your data frame such that it starts with the smallest planet and ends with the largest one. A sort on the diameter column.

In [34]:
```
planets_df

# Use order() to create positions
positions <-  order(planets_df$diameter)
positions

# Use positions to sort planets_df
planets_df[positions, ]
```

name	type	diameter	rotation	rings
Mercury	Terrestrial planet	0.382	58.64	FALSE
Venus	Terrestrial planet	0.949	-243.02	FALSE
Earth	Terrestrial planet	1.000	1.00	FALSE
Mars	Terrestrial planet	0.532	1.03	FALSE
Jupiter	Gas giant	11.209	0.41	TRUE
Saturn	Gas giant	9.449	0.43	TRUE
Uranus	Gas giant	4.007	-0.72	TRUE
Neptune	Gas giant	3.883	0.67	TRUE

1 4 2 3 8 7 6 5

	name	type	diameter	rotation	rings
1	Mercury	Terrestrial planet	0.382	58.64	FALSE
4	Mars	Terrestrial planet	0.532	1.03	FALSE
2	Venus	Terrestrial planet	0.949	-243.02	FALSE
3	Earth	Terrestrial planet	1.000	1.00	FALSE
8	Neptune	Gas giant	3.883	0.67	TRUE
7	Uranus	Gas giant	4.007	-0.72	TRUE
6	Saturn	Gas giant	9.449	0.43	TRUE
5	Jupiter	Gas giant	11.209	0.41	TRUE

1.3.5 Lists

A list in R is similar to your to-do list at work or school: the different items on that list most likely differ in length, characteristic, and type of activity that has to be done.

A list in R allows you to gather a variety of objects under one name (that is, the name of the list) in an ordered way. These objects can be matrices, vectors, data frames, even other lists, etc. It is not even required that these objects are related to each other in any way.

You could say that a list is some kind super data type: you can store practically any piece of information in it!

1.3.5.1 Creating a list

Let us create our first list! To construct a list you use the function **list()**

my_list <- list(comp1, comp2 ...)

The arguments to the list function are the list components. Remember, these components can be matrices, vectors, other lists, …

In [1]:
```
# Vector with numerics from 1 up to 10
my_vector <- 1:10

# Matrix with numerics from 1 up to 9
my_matrix <- matrix(1:9, ncol = 3)

# First 10 elements of the built-in data frame mtcars
my_df <- mtcars[1:10,]

# Construct list with these different elements:
my_list <- list(my_vector, my_matrix, my_df)
my_list
```

1. 1 2 3 4 5 6 7 8 9 10
2. 1 4 7
   2 5 8
   3 6 9
3.

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4

1.3.5.2 Creating a named list

Just like on your to-do list, you want to avoid not knowing or remembering what the components of your list stand for. That is why you should give names to them:

my_list <- list(name1 = your_comp1, name2 = your_comp2)

This creates a list with components that are named name1, name2, and so on. If you want to name your lists after you've created them, you can use the **names()** function as you did with vectors. The following commands are fully equivalent to the assignment above:

Change the code of the previous exercise by adding names to the components. Use for my_vector the name vec, for my_matrix the name mat and for my_df the name df.

Print out my_list so you can inspect the output.

In [3]:
```
# Vector with numerics from 1 up to 10
my_vector <- 1:10

# Matrix with numerics from 1 up to 9
my_matrix <- matrix(1:9, ncol = 3)

# First 10 elements of the built-in data frame mtcars
my_df <- mtcars[1:10,]

# Adapt list() call to give the components names
my_list <- list(my_vector, my_matrix, my_df)

#name the list
names(my_list) <- c("vec", "mat", "df")

# Print out my_list
my_list
```

$vec

1 2 3 4 5 6 7 8 9 10

$mat

1	4	7
2	5	8
3	6	9

$df

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4

1.3.5.3 Selecting elements from a list

Your list will often be built out of numerous elements and components. Therefore, getting a single element, multiple elements, or a component out of it is not always straightforward.

One way to select a component is using the numbered position of that component. For example, to "grab" the first component of shining_list you type

my_list[[1]]

A quick way to check this out is typing it in the console. Important to remember: to select elements from vectors, you use single square brackets: [ ]. Don't mix them up!

You can also refer to the names of the components, with [[ ]] or with the $ sign. Both will select the data frame representing the reviews:

In [4]:
```
my_list[["df"]]
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4

In [5]:
```
my_list$df
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4

Besides selecting components, you often need to select specific elements out of these components

In [6]:
```
my_list[["df"]][1,]
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21	6	160	110	3.9	2.62	16.46	0	1	4	4

In [7]:
```
my_list$df[1,]
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21	6	160	110	3.9	2.62	16.46	0	1	4	4

In [ ]:
```

```

In [8]:
```
movie_title <- c("The Departed")

movie_actors <- c("Leonardo DiCaprio",
                  "Matt Damon",
                  "Jack Nicholson",
                  "Mark Wahlberg",
                  "Vera Farmiga",
                  "Martin Sheen" )

scores <- c(4.6, 5, 4.8, 5, 4.2)

comments <- c("I would watch it again",
              "Amazing!",
              "I liked it",
              "One of the best movies",
              "Fascinating plot")

# Save the average of the scores vector as avg_review
avg_review = mean(scores)
avg_review

# Combine scores and comments into the reviews_df data frame
reviews_df <- data.frame(scores, comments)
reviews_df

# Create and print out a list, called departed_list
departed_list <- list(movie_title, movie_actors, reviews_df, avg_review)
departed_list
```

4.72

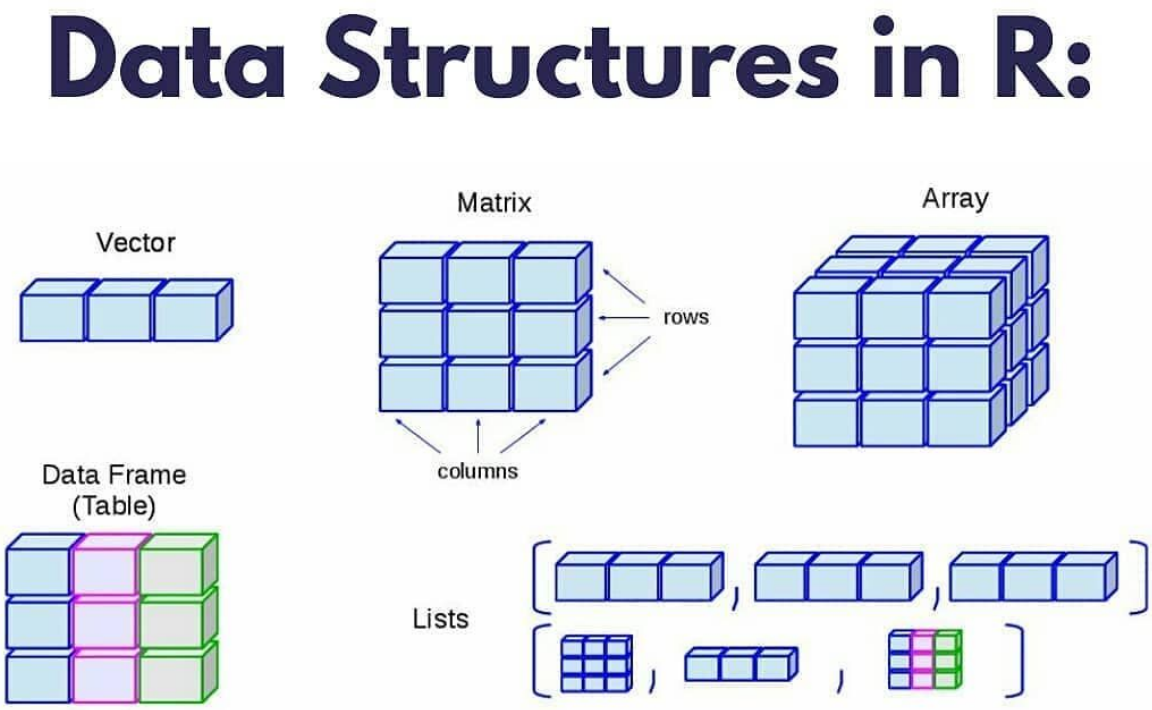
scores	comments
4.6	I would watch it again
5.0	Amazing!
4.8	I liked it
5.0	One of the best movies
4.2	Fascinating plot

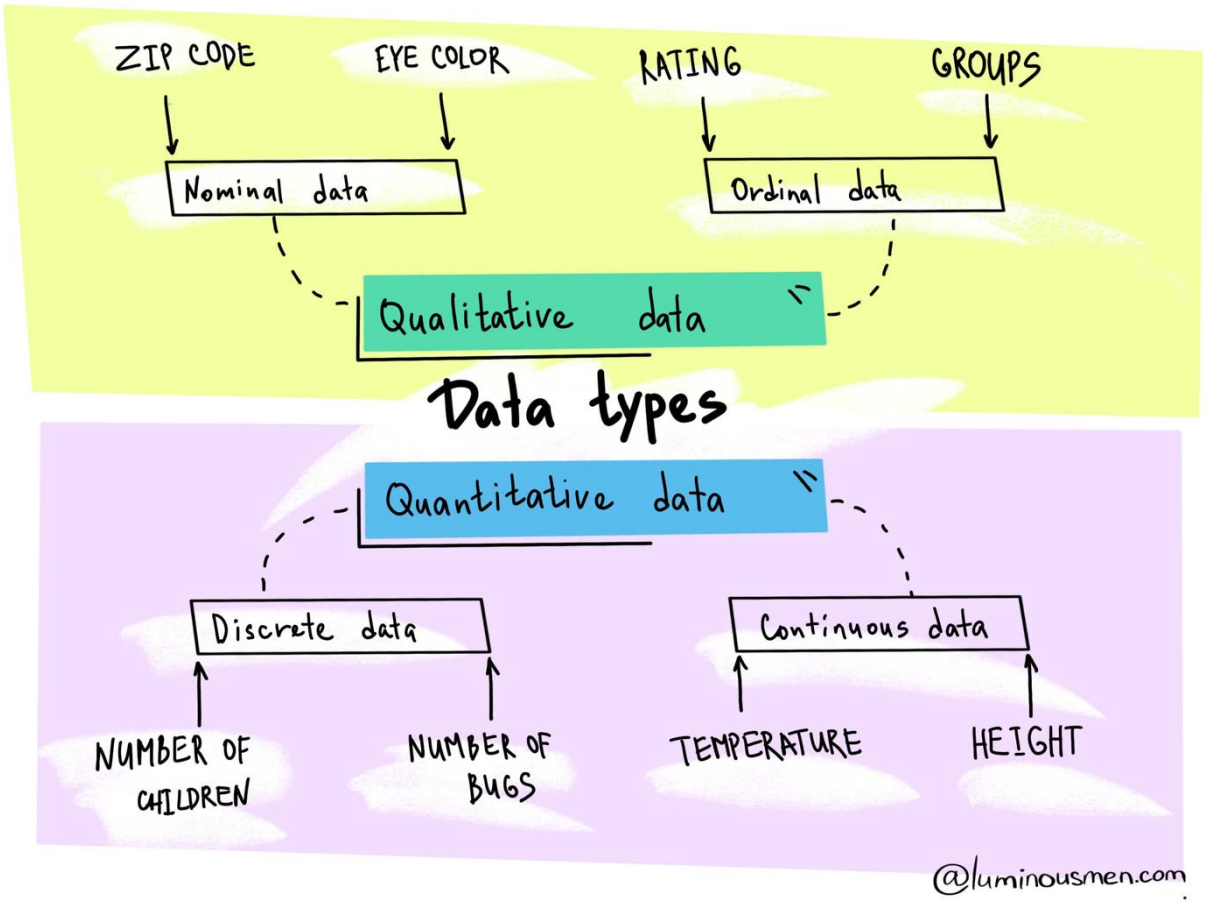
1. 'The Departed'
2. 'Leonardo DiCaprio' 'Matt Damon' 'Jack Nicholson' 'Mark Wahlberg' 'Vera Farmiga' 'Martin Sheen'
3.

scores	comments
4.6	I would watch it again
5.0	Amazing!
4.8	I liked it
5.0	One of the best movies
4.2	Fascinating plot

4. 4.72

1.3.6 Summarizing data types





2 Intermediate R

2.1 Conditionals and Control Flow

Relational operators, or comparators are operators which help us see how one R object relates to another. For example, you can check whether two objects are equals with "=="

In [2]:
```
TRUE == TRUE
```

TRUE

In [3]:
```
"hello" == "bye"
```

FALSE

The opposite of the equality operator is the inequality "!="

In [4]:
```
"hello" != "bye"
```

TRUE

Of course, there are also cases where you need more than simply equality and inequality operators. What about checking if an R object is less than or greater than anoter R object?

In [5]:
```
3 < 5
```

TRUE

In [6]:
```
5 > 3
```

TRUE

For numerics this make sense, but how would this work for character strings and logical values?

In [7]:
```
"Hello" > "GoodBye"
```

TRUE

Why is Hello greater than GoodBye? It's because R uses alphabet to sort letter strings, sinces "H" comes after to "G" in the alphabet, then Hello > GoodBye.

How about logical values? Is TRUE less than FALSE?

In [8]:
```
TRUE < FALSE
```

FALSE

That's because, TRUE correspond to 1 and FALSE to 0.

"greater or equal than" .....or..... "less or equal than"

In [9]:
```
5 >= 3
```

TRUE

2.1.1 Relational operators and vectors

In [11]:
```
linkedin <- c(16, 9, 13, 5, 2, 17, 14)
linkedin
```

16 9 13 5 2 17 14

In [12]:
```
linkedin > 10
```

TRUE FALSE TRUE FALSE FALSE TRUE TRUE

In [13]:
```
facebook <- c(17, 7, 5, 16, 8, 13, 14)
facebook
```

17 7 5 16 8 13 14

In [14]:
```
facebook < linkedin
```

FALSE TRUE TRUE FALSE FALSE TRUE FALSE

In this case, the comparison is done for every element of the vector, one by one.

In [10]:
```
# Comparison of numerics
-6*14 != 17-101

# Comparison of character strings
"useR" != "use"

# Compare a logical with a numeric
TRUE == 1

# Comparison of numerics
-6 * 5 + 2 > -10 + 1

# Comparison of character strings
"raining" <= "raining dogs"

# Comparison of logicals
TRUE > FALSE
```

FALSE

TRUE

TRUE

FALSE

TRUE

TRUE

2.1.1.1 Compare matrices

R's ability to deal with different data structures for comparisons does not stop at vectors. Matrices and relational operators also work together seamlessly.

Instead of in vectors, the LinkedIn and Facebook data (number of views per day) is now stored in a matrix called views. The first row contains the LinkedIn information; the second row the Facebook information. The original vectors facebook and linkedin are still available as well.

In [11]:
```
# The social data has been created
linkedin <- c(16, 9, 13, 5, 2, 17, 14)
facebook <- c(17, 7, 5, 16, 8, 13, 14)

views <- matrix(c(linkedin, facebook), nrow = 2, byrow = TRUE)
views
```

16	9	13	5	2	17	14
17	7	5	16	8	13	14

When were the views exactly equal to 13? Use the views matrix to return a logical matrix.

In [12]:
```
views == 13
```

FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE

When is views less than or equal to 14?

In [23]:
```
views <= 14
```

A matrix: 2 × 7 of type lgl

FALSE	TRUE	TRUE	TRUE	TRUE	FALSE	TRUE
FALSE	TRUE	TRUE	FALSE	TRUE	TRUE	TRUE

In [13]:
```
names <- c("mon", "tue", "wen", "thu", "fri", "sat", "sun")
names
colnames(views) <- names
views

views <= 14
```

'mon' 'tue' 'wen' 'thu' 'fri' 'sat' 'sun'

mon	tue	wen	thu	fri	sat	sun
16	9	13	5	2	17	14
17	7	5	16	8	13	14

mon	tue	wen	thu	fri	sat	sun
FALSE	TRUE	TRUE	TRUE	TRUE	FALSE	TRUE
FALSE	TRUE	TRUE	FALSE	TRUE	TRUE	TRUE

2.1.2 Logical operators

2.1.2.1 AND operator "&"

The **AND operator (&)** works just as you would expect. It typically takes two logical values and returns TRUE only if both the logical values are TRUE themselves. This means that

TRUE and TRUE evaluates to TRUE,

but that FALSE and TRUE, TRUE and FALSE and FALSE and FALSE all evaluate to FALSE.

Instead of using logical values, we can of course use the results of comparisons.

Suppose we have a variable x, equal to 12. To check if this variable is greater than 5 but less than 15, we can use x greater than 5 and x less than 15. As you already learned, the first part will evaluate to TRUE. The second part, will also evaluate to TRUE. So the result of this expression is TRUE. This makes sense, because 12 lies between 5 and 15. However, if x were equal to 17, the expression x greater than 5 & x less than 15 would simplify to TRUE and FALSE. which results in this expression being FALSE.

In [39]:
```
x <- 12
x >= 5 & x <= 15

x <- 17
x >= 5 & x <= 15
```

TRUE

FALSE

2.1.2.2 OR operator "|"

The **OR operator (|)** works similarly, but the difference is that only at least one of the logical values it uses should be equal to TRUE for the entire OR operation to evaluate to TRUE. This means that,

TRUE or TRUE equals TRUE,

but that also TRUE or FALSE and FALSE or TRUE evaluate to TRUE.

When both logicals are FALSE in an OR operation, so in the case of FALSE or FALSE, the result is FALSE.

Remember that the OR operation is not an exclusive or operation, so TRUE or TRUE equals TRUE as well.

Just as for AND operators, we can use comparisons together with the OR operator. Suppose we have a variable y, equal to 4 this time. To see if this variable is less than 5 or greater than 15, we can use this expression. R will first carry out the comparisons, resulting in TRUE or FALSE, which in turn results in TRUE. Now, let's have y equal 14. The expression y less than 5 or y greater than 15 now evaluates to FALSE or FALSE. Neither one of the comparisons are TRUE, so the result is FALSE.

In [41]:
```
y <- 4

y <= 4 | y >= 15

y <- 14

y <= 4 | y >= 15
```

TRUE

FALSE

2.1.2.3 NOT operator "!"

The **NOT operator (|)**, represented by an exclamation mark, simply negates the logical value it's used on. So exclamation mark TRUE evaluates to FALSE, while exclamation mark FALSE evaluates to TRUE. Just as the OR and AND operators, you can use the NOT operator in combination with logical operators. This is not always necessary, however, because this line of code is exactly the same as this one.

In [42]:
```
!TRUE
```

FALSE

In [45]:
```
z <- 100

!(z < 5)

z >= 5
```

TRUE

TRUE

In [49]:
```
is.numeric(5)

!is.numeric(5)

is.numeric("Hello")

!is.numeric("Hello")
```

TRUE

FALSE

FALSE

TRUE

2.1.2.4 Logical operators and vectors

In [55]:
```
c(TRUE, TRUE, FALSE) & c(TRUE, FALSE, FALSE)

c(TRUE, TRUE, FALSE) | c(TRUE, FALSE, FALSE)

!c(TRUE, TRUE, FALSE)
```

TRUE FALSE FALSE

TRUE TRUE FALSE

FALSE FALSE TRUE

2.1.2.5 "&" vs "&&", "|" vs "||"

The difference between a single and a double ampersand or vertical bar. In R you can use both the single sign version or the double sign version, but the result of the logical operation you're carrying out can be different. The biggest difference occurs when you use the two types of operations on vectors. As we've seen before, this expression, evaluates to a vector containing TRUE, FALSE and FALSE. However, if we use a double ampersand, we simply get TRUE. That's because the double ampersand operation only examines the first element of each vector. In this case the first elements are TRUE and TRUE, so the expression returns TRUE.

In [57]:
```
c(TRUE, TRUE, FALSE) & c(TRUE, FALSE, FALSE)

c(TRUE, TRUE, FALSE) && c(TRUE, FALSE, FALSE)


c(TRUE, TRUE, FALSE) | c(TRUE, FALSE, FALSE)

c(TRUE, TRUE, FALSE) || c(TRUE, FALSE, FALSE)
```

TRUE FALSE FALSE

TRUE

TRUE TRUE FALSE

TRUE

In [15]:
```
# The linkedin and last variable
linkedin <- c(16, 9, 13, 5, 2, 17, 14)
last <- tail(linkedin, 1)
last
```

14

Is last under 5 or above 10?

In [16]:
```
last < 5 | last > 10
```

TRUE

Is last between 15 (exclusive) and 20 (inclusive)?

In [17]:
```
last > 15 & last <=20
```

FALSE

In [18]:
```
# The social data has been created
linkedin <- c(16, 9, 13, 5, 2, 17, 14)
facebook <- c(17, 7, 5, 16, 8, 13, 14)

views <- matrix(c(linkedin, facebook), nrow = 2, byrow = TRUE)
views
```

16	9	13	5	2	17	14
17	7	5	16	8	13	14

When did LinkedIn views exceed 10 and did Facebook views fail to reach 10 for a particular day?

In [19]:
```
linkedin > 10 & facebook < 10
```

FALSE FALSE TRUE FALSE FALSE FALSE FALSE

When were one or both of your LinkedIn and Facebook profiles visited at least 12 times?

In [68]:
```
linkedin >=12 | facebook >=12
```

TRUE FALSE TRUE TRUE FALSE TRUE TRUE

When is the views matrix equal to a number between 11 and 14, excluding 11 and including 14?

In [20]:
```
views > 11 & views <=14
```

FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE
FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE

In [21]:
```
x <- 5
y <- 7
!(!(x < 4) & !!!(y > 12))
```

FALSE

2.2 Conditional Statments

2.2.1 if statement

The if statement takes a condition; if the condition evaluates to TRUE, the R code associated with the if statement is executed. The condition to check appears inside parentheses, while the R code that has to be executed if the condition is TRUE, follows in curly brackets.

if(condition) {

    expr

}

In [22]:
```
x <- -3

if(x < 0){
    print("x is a negative number")
}
```

```
[1] "x is a negative number"
```

2.2.2 else statement

In [23]:
```
x <- 3

if(x < 0){
    print("x is a negative number")
} else {
    print("x is a positive number")
}
```

```
[1] "x is a positive number"
```

2.2.3 else if statement

In [24]:
```
x <- 0

if(x < 0){
    print("x is a negative number")
} else if( x == 0){
    print("x is zero")
} else {
    print("x is a positive number")
}
```

```
[1] "x is zero"
```

Remember that as soon as R stumbles upon a condition that evaluates to TRUE, R executes the corresponding code and then ignores the rest of the control structure. This becomes important if the conditions you list are not mutually exclusive.

In [25]:
```
x <- 6

if(x %% 2 == 0) {
    print("divisible by 2")
} else if(x %% 3 == 0) {
    print("divisible by 3")
} else {
    print("not divisible by 2 or 3")
}
```

```
[1] "divisible by 2"
```

Remember your vectors with social profile views? Let's look at it from another angle. The medium variable gives information about the social website; the num_views variable denotes the actual number of views that particular medium had on the last day of your recordings. Both variables have been pre-defined for you.

In [27]:
```
# Variables related to your last day of recordings
medium <- "LinkedIn"
num_views <- 14
```

Examine the if statement that prints out "Showing LinkedIn information" if the medium variable equals "LinkedIn".

In [28]:
```
if(medium == "LinkedIn"){
    print("Showing LinkedIn information")
}
```

```
[1] "Showing LinkedIn information"
```

Code an if statement that prints "You are popular!" to the console if the num_views variable

exceeds 1500.

In [29]:
```
if(num_views > 1500){
    print("You are popular!")
}
```

Add an else statement to both control structures, such that

- "Unknown medium" gets printed out to the console when the if-condition on medium does not hold.
- R prints out "Try to be more visible!" when the if-condition on num_views is not met.

In [30]:
```
if(medium == "LinkedIn"){
    print("Showing LinkedIn information")
}else {
    print("Unknown medium")
}
```

```
[1] "Showing LinkedIn information"
```

In [31]:
```
if(num_views > 1500){
    print("You are popular!")
}else {
    print("Try to be more visible!")
}
```

```
[1] "Try to be more visible!"
```

Add code to both control structures such that:

- R prints out "Showing Facebook information" if medium is equal to "Facebook". Remember that R is case sensitive!
- "Your number of views is average" is printed if num_views is between 15 (inclusive) and 10 (exclusive).

In [32]:
```
if(medium == "LinkedIn"){
    print("Showing LinkedIn information")
}else if(medium == "Facebook"){
    print("Showing Facebook information")
}else {
    print("Unknown medium")
}
```

```
[1] "Showing LinkedIn information"
```

In [33]:
```
# Control structure for num_views
if (num_views > 15) {
  print("You're popular!")
} else if (num_views <= 15 & num_views > 10) {
  print("Your number of views is average")
} else {
  print("Try to be more visible!")
}
```

```
[1] "Your number of views is average"
```

In [34]:
```
number <- 2500
if (number < 10) {
  if (number < 5) {
    result <- "extra small"
  } else {
    result <- "small"
  }
} else if (number < 100) {
  result <- "medium"
} else {
  result <- "large"
}
print(result)
```

```
[1] "large"
```

2.3 Loops

2.3.1 While loop

The syntax of a while loop is pretty similar to the if statement, as you can see here. Let's have a look at a very simple example: we'll simply make R increment a counter until it reaches the value 7.

In [36]:
```
# ?paste

ctr <- 0

while(ctr <= 7){
    print(paste("ctr is set to:", ctr))
    ctr <- ctr + 1
}
```

```
[1] "ctr is set to: 0"
[1] "ctr is set to: 1"
[1] "ctr is set to: 2"
[1] "ctr is set to: 3"
[1] "ctr is set to: 4"
[1] "ctr is set to: 5"
[1] "ctr is set to: 6"
[1] "ctr is set to: 7"
```

The line of code to increment ctr is crucial, suppose to remove this line, what's happen?

?paste

ctr <- 0

while(ctr <= 7){ print(paste("ctr is set to:", ctr)) }

The **break** statement simply breaks out of the while loop.

In [37]:
```
ctr <- 0

while(ctr <= 7){
    if(ctr == 3){
        break
    }
    print(paste("ctr is set to:", ctr))
    ctr <- ctr + 1
}
```

```
[1] "ctr is set to: 0"
[1] "ctr is set to: 1"
[1] "ctr is set to: 2"
```

2.3.2 For loops

The for loop is somewhat different from the while loop. This can be read as: for each var, a variable, in seq, a sequence, execute expressions.

for(var in seq){

    expr

}

Makes sense? Let's see how this actually works with an example.

In [38]:
```
cities <- c("New York", "Paris",
            "London", "Tokyo",
            "Rio de Janeiro", "Talca")

#we can print cities
cities
```

'New York' 'Paris' 'London' 'Tokyo' 'Rio de Janeiro' 'Talca'

But suppose we want to have a different printout for every element in the vector

In [39]:
```
for(city in cities){
    print(city)
}
```

```
[1] "New York"
[1] "Paris"
[1] "London"
[1] "Tokyo"
[1] "Rio de Janeiro"
[1] "Talca"
```

How does R handle this code? At the start of the loop, R evaluates the seq element, being cities in our case. It realizes that it is a vector containing 6 elements. Next, R stores the first element of this sequence in the variable city, so city equals "New York" now.

Then, the expression, print(city), is executed, printing out "New York" to the console.

After the execution, R stores the second element of the cities vector, "Paris", in city and re-runs the code. This process repeats itself until all cities in the cities vector are iterated over.

The for loop does not only work on vectors: it also works with lists for example.

In [40]:
```
cities <- list("New York", "Paris",
            "London", "Tokyo",
            "Rio de Janeiro", "Talca")
cities
```

1. 'New York'
2. 'Paris'
3. 'London'
4. 'Tokyo'
5. 'Rio de Janeiro'
6. 'Talca'

In [41]:
```
for(city in cities){
    print(city)
}
```

```
[1] "New York"
[1] "Paris"
[1] "London"
[1] "Tokyo"
[1] "Rio de Janeiro"
[1] "Talca"
```

The exact same for loop as we've been using before can be used for lists, and the result is exactly the same. So there's no need to worry about the difference between subsetting vectors and lists, because the for loop does this for us.

The **break** statement is a statement that you already know: just like in the while loop, break in a for loop simply stops the execution of the code and abandons the for loop altogether.

In [43]:
```
cities <- list("New York", "Paris",
               "London", "Tokyo",
               "Rio de Janeiro", "Talca")
cities

for(city in cities){
    if(nchar(city) == 6){
        break
    }
    print(city)
}
```

1. 'New York'
2. 'Paris'
3. 'London'
4. 'Tokyo'
5. 'Rio de Janeiro'
6. 'Talca'

```
[1] "New York"
[1] "Paris"
```

The **next** statement also alters the flow of your for loop, but does so in a slightly different way.

In [44]:
```
cities <- list("New York", "Paris",
               "London", "Tokyo",
               "Rio de Janeiro", "Talca")
cities

for(city in cities){
    if(nchar(city) == 6){
        next
    }
    print(city)
}
```

1. 'New York'
2. 'Paris'
3. 'London'
4. 'Tokyo'
5. 'Rio de Janeiro'
6. 'Talca'

```
[1] "New York"
[1] "Paris"
[1] "Tokyo"
[1] "Rio de Janeiro"
[1] "Talca"
```

accessing to each index

In [46]:
```
cities <- c("New York", "Paris",
            "London", "Tokyo",
            "Rio de Janeiro", "Talca")

for(i in 1:length(cities)){
    print(cities[i])
}
```

```
[1] "New York"
[1] "Paris"
[1] "London"
[1] "Tokyo"
[1] "Rio de Janeiro"
[1] "Talca"
```

In [16]:
```
cities <- c("New York", "Paris",
            "London", "Tokyo",
            "Rio de Janeiro", "Talca")

for(i in 1:length(cities)){
    print(paste(cities[i], "is on position", i, "in the cities vector"))
}
```

```
[1] "New York is on position 1 in the cities vector"
[1] "Paris is on position 2 in the cities vector"
[1] "London is on position 3 in the cities vector"
[1] "Tokyo is on position 4 in the cities vector"
[1] "Rio de Janeiro is on position 5 in the cities vector"
[1] "Talca is on position 6 in the cities vector"
```

for in matrix

In [50]:
```
a <- c("O", "NA", "X")
b <- c("NA", "O", "O")
c <- c("X", "NA", "X")
d <- c(a, b, c)

ttt <- matrix(d,  byrow = T, nrow = 3)
ttt


for (i in 1:nrow(ttt)) {
  for (j in 1:ncol(ttt)) {
    print(paste("On row", i, "and column", j, "the board contains", ttt[i,j]))
  }
}
```

```
  O  NA  X
 NA   O  O
  X  NA  X

[1] "On row 1 and column 1 the board contains O"
[1] "On row 1 and column 2 the board contains NA"
[1] "On row 1 and column 3 the board contains X"
[1] "On row 2 and column 1 the board contains NA"
[1] "On row 2 and column 2 the board contains O"
[1] "On row 2 and column 3 the board contains O"
[1] "On row 3 and column 1 the board contains X"
[1] "On row 3 and column 2 the board contains NA"
[1] "On row 3 and column 3 the board contains X"
```

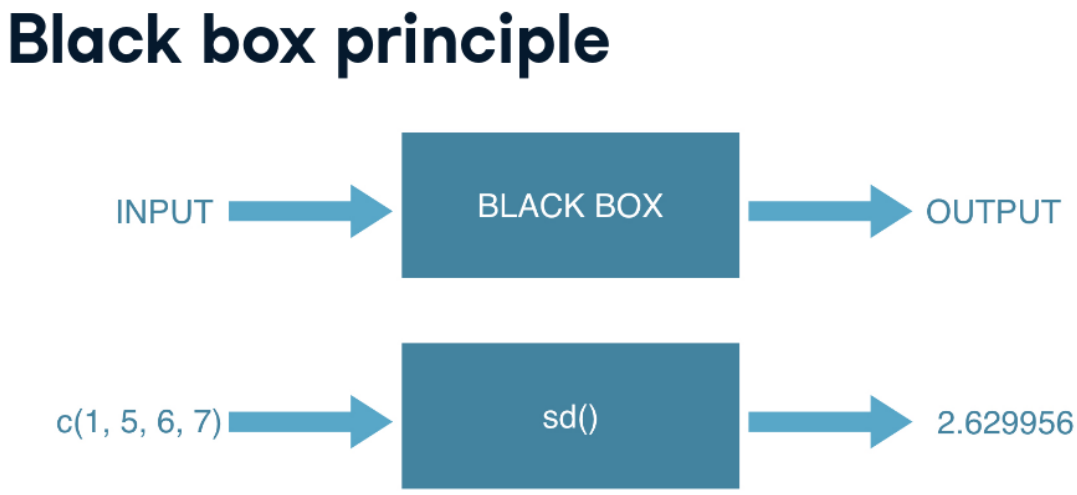
2.4 Functions

Not surprisingly, you've already used functions before. Remember the time you created a list? You used the **list()** function. Or the time you wanted to display the contents of a variable? You used the

print() function. But what are functions and how do they work?

You can think of a function as some kind of black box. You give an input to the black box, the black box processes this input and it returns some output

The R **sd** function calculates the standard deviation of a vector. Our black box in this case is the sd function.



In [3]:
```
sd(c(1,5,6,7))
```

2.62995563967658

How one specific function works?

- **help(sd)**
- **?sd**

In [4]:
```
?sd
```

Function documentation presents a lot of information. If we have a look at the "Usage" section, we see that the sd function actually takes two arguments, x and na.rm

In [5]:
```
sd(c(1,5,6,7, NA))
```

<NA>

In [6]:
```
sd(c(1,5,6,7), na.rm = T)
```

2.62995563967658

In [7]:
```
sd(c(1,5,6,7), T)
```

2.62995563967658

2.4.1 The args function

This is a function to learn about the arguments of a function without having to read through the entire documentation

In [8]:
```
args(sd)
```

```
function (x, na.rm = FALSE)
NULL
```

In [9]:
```
?mean
```

Default S3 method:

mean(x, trim = 0, na.rm = FALSE, ...)

In [10]:
```
# The linkedin and facebook vectors have already been created for you
linkedin <- c(16, 9, 13, 5, 2, 17, 14)
facebook <- c(17, 7, 5, 16, 8, 13, 14)

# Calculate the mean of the sum
avg_sum <- mean(linkedin + facebook)

# Calculate the trimmed mean of the sum
avg_sum_trimmed <- mean(linkedin + facebook, trim=0.2)

# Inspect both new variables
avg_sum
avg_sum_trimmed
```

22.2857142857143

22.6

The sd() function has an optional argument, na.rm that specified whether or not to remove missing values from the input vector before calculating the standard deviation.

If you've had a good look at the documentation, you'll know by now that the mean() function also has this argument, na.rm, and it does the exact same thing. By default, it is set to FALSE, as the Usage of the default S3 method shows:

In [11]:
```
# The linkedin and facebook vectors have already been created for you
linkedin <- c(16, 9, 13, 5, NA, 17, 14)
facebook <- c(17, NA, 5, 16, 8, 13, 14)

# Basic average of linkedin
mean(linkedin)

# Advanced average of linkedin
mean(linkedin, 0, T)
```

<NA>

12.3333333333333

2.4.2 Functions inside functions

You already know that R functions return objects that you can then use somewhere else. This makes it easy to use functions inside functions, as you've seen before:

speed <- 31

print(paste("Your speed is", speed))

In [12]:
```
# The linkedin and facebook vectors have already been created for you
linkedin <- c(16, 9, 13, 5, NA, 17, 14)
facebook <- c(17, NA, 5, 16, 8, 13, 14)

# Calculate the mean absolute deviation
mean(abs(linkedin-facebook), 0, T)
```

4.8

In [14]:
```
?read.table
```

In [15]:
```
read.table("myfile.txt", "-", TRUE)
```

```
Warning message in file(file, "rt"):
“cannot open file 'myfile.txt': No such file or directory”

Error in file(file, "rt"): cannot open the connection
Traceback:

1. read.table("myfile.txt", "-", TRUE)
2. file(file, "rt")
```

In [16]:
```
read.table("myfile.txt", sep = "-", header = TRUE)
```

```
Warning message in file(file, "rt"):
“cannot open file 'myfile.txt': No such file or directory”

Error in file(file, "rt"): cannot open the connection
Traceback:

1. read.table("myfile.txt", sep = "-", header = TRUE)
2. file(file, "rt")
```

2.4.3 Writing functions

A function typically serves a particular need or solves a particular problem, without having to care about how the function does this. Remember the 'black-box principle'

my_func <- function(arg1, arg2){

body

}

In [19]:
```
triple <- function(x){
    x * 3
}

ls()
```

'a' 'avg_sum' 'avg_sum_trimmed' 'b' 'facebook' 'linkedin' 'triple' 'var_dic_list'

You see that triple function has been created in the workspace

In [20]:
```
triple(6)
```

18

In [21]:
```
triple <- function(x){
    y <- x * 3
    return(y)
}

triple(6)
```

18

2.4.4 Optional argument

In [23]:
```
math_magic <- function(a, b){
    a*b + a/b
}

math_magic(2,3)
math_magic(2)
```

6.66666666666667

```
Error in math_magic(2): argument "b" is missing, with no default
Traceback:

1. math_magic(2)
```

In [24]:
```
math_magic <- function(a, b=1){
    a*b + a/b
}

math_magic(2)
```

4

but, what's happen when we use x, and 0

In [25]:
```
math_magic <- function(a, b=1){
    a*b + a/b
}

math_magic(2,0)
```

Inf

That's cause 2/0 in infinite!, so:

In [27]:
```
math_magic <- function(a, b=1){

    if(b == 0){
        return(0)
    }
    a*b + a/b
}

math_magic(2,0)
```

0

Note that **return** function is similar to **break**

There are situations in which your function does not require an input. Let's say you want to write a function that gives us the random outcome of throwing a fair die:

In [29]:
```
throw_die <- function() {
  number <- sample(1:6, size = 1)
  number
}

throw_die()
```

3

Function scoping implies that variables that are defined inside a function are not accessible outside that function

In [32]:
```
pow_two <- function(x) {
  y <- x ^ 2
  return(y)
}
pow_two(4)

y

x
```

16

```
Error in eval(expr, envir, enclos): object 'y' not found
Traceback:
```

In [36]:
```
triple <- function(x) {
  x <- 3*x
  x
}
a <- 5
triple(a)
a
```

15

5

2.5 R Packages

R packages are bundles of code, data, documentation, and tests that are easy to share with others. For example, the **mean**, **list** and **sample** functions are all part of the base package, which contains the basic functionality to use R. Another example package, specifically for data

visualization, is the **ggvis package**. It contains functions such as layer_points, scale_nominal and add_axis.

In [76]:
```
#install.packages("ggvis")

head(mtcars)
str(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
'data.frame':   32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
 $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

In [77]:
```
library(ggvis)

mtcars %>% ggvis(~disp, ~mpg) %>% layer_points()
```

```
Warning message:
"package 'ggvis' was built under R version 3.6.3"
```

2.5.1 Install packages

Before you can use a package, you will first have to install it. The base package is automatically installed when you install R. The ggvis package on the other hand won't come with the bundled R installation. But fear not! You can easily install it:

install.packages("ggvis")

This function goes to CRAN. CRAN is short for the Comprehensive R Archive Network, a repository where thousands of packages are available. The function downloads the package file and installs the package on your system.

2.5.2 Load packages

To do that, we'll have to actually load the package into our current R session. When R loads a package, it actually attaches it to the search list. This is a list of packages and environments that R looks through to find the variable or function you want to use. To have a look at this list, you can use the search() function.

In [10]:
```
search()
```

'.GlobalEnv' 'package:ggvis' 'package:jsonlite' 'jupyter:irkernel' 'package:stats' 'package:graphics' 'package:grDevices' 'package:utils' 'package:datasets' 'package:methods' 'Autoloads' 'package:base'

Whenever you execute code that depends on any other variable or function, R goes through all these packages one after the other to find it. Apart from all the packages that are loaded into our R session, we also see ".GlobalEnv"; this is our own workspace, where the user-defined R objects live.

You can also see that the search() path already contains a bunch of packages. When R is started, it loads 7 packages on default, among which is the base package. Others are the utils, datasets and methods package. That's why, when you start R, you can use the mean function, or the install.packages function without having to explicitly load the package.

The ggvis package that we've installed earlier, however, won't be loaded automatically. If we try to access the ggvis function, for example, R will return an error, telling us that the function ggvis could not be found. That's because ggvis is not yet in the search list.

2.5.2.1 Load packages: library()

To access ggvis' functionality, we'll have to load the package using the library command. This command takes the name of the package and adds the package to the search list, right after the global environment, making all the functions, data and pre-compiled code it contains available and ready to be used.

library(ggvis)

2.5.2.2 Load packages: require()

Just like the **library()** function, require loads packages into your R session. The only difference appears when you're trying to load a package that is not yet installed. Let's say you want to load the **data.table package**, a package to perform data manipulation, but that this package is not yet

installed. If you call **library(data.table)**. R throws an error in this case. However, when you execute **require(data.table)**, you get a warning.

2.6 Apply family

2.6.1 lapply

When you're using R, you will often encounter vectors and lists containing all sorts of information. You've seen before that the for loop is there to help us iterate over all kinds of data structures. But I'm glad to tell you there's an even easier way!

Let's have a look at some information related to New York City. It is stored in a list.

In [17]:
```
nyc <- list(pop = 8405837,
            boroughs = c("Manhattan", "Bronx", "Brooklyn", "Queens", "Staten Islar
            capital = FALSE)
```

Suppose we want to find out the class of each element of this list. As you already know, you could use a for loop that iterates over the different elements of nyc, and call the **class() function**.

In [18]:
```
for(info in nyc){
    print(class(info))
}
```

```
[1] "numeric"
[1] "character"
[1] "logical"
```

As expected, the first element is numeric, the second one is a character string, and the third is a logical. This however is quite a bit of code just to find out the class of elements of a list, isn't it?

lapply is here to rescue us from writing too much code for simple tasks such as these.

In [19]:
```
lapply(nyc, class)
```

$pop
'numeric'
$boroughs
'character'
$capital
'logical'

lapply iterated over the inputted list, nyc, and on each element of the list, applied the function class(). The output of lapply includes the results of calling the class function over each of the list

elements. The first element in the output now contains the class of the first element in nyc, namely "numeric".

The second element in the output contains the class of the second element in nyc, "character", and the third element contains logical, the class of the third input element. Also notice that the names of the list, pop, boroughs and capital are maintained here, that's pretty useful

Let's have a look at another example, now with a vector of city names.

In [20]:
```
cities <- c("New York", "Paris", "london", "Tokyo", "Rio de Janeiro", "Talca")
```

Suppose we want to build a vector of the same length as cities, containing the number of characters of each city name. We can do that by using the following for loop. First, we create an empty vector, called num_chars. Inside the for loop, using a looping index, we gradually fill this vector up using the nchar function. We get the wanted result, but it's a lot of code to do a simple thing.

In [21]:
```
num_chars <- c()

for(i in 1:length(cities)){
    num_chars[i] <- nchar(cities[i])
}

num_chars
```

8 5 6 5 14 5

Fortunately, refactoring this code to use lapply is easy. As the input, we use the cities vector as well as the function we want to apply on each element in this vector: We get a list containing the number of characters of the corresponding cities in the input vector. Notice here that the output is a list, although the input was a vector. This is something important about the lapply function: it always returns a list, irrespective of the input data structure.

In [23]:
```
lapply(cities, nchar)
```

1. 8
2. 5
3. 6
4. 5
5. 14
6. 5

If you want to convert this list to a vector, you can simply wrap this lapply function inside the unlist() function which turns a list into a vector.

In [24]:
```
unlist(lapply(cities, nchar))
```

```
8 5 6 5 14 5
```

In a single one-liner, we have achieved the exact same thing as the bulky for loop. Moreover, by using lapply our code looks much more intuitive and readable.

In the previous examples we applied the functions class() and nchar() on every element of the input vector or list, but lapply can also be used with functions that you've written yourself.

In [32]:
```
oil_prices <- list(2.37, 2.49, 2.18, 2.47, 2.32, 2.00, 1.99)

triple <- function(x){
    3 * x
}

result <- lapply(oil_prices, triple)

result

str(result)
```

1. 7.11
2. 7.47
3. 6.54
4. 7.41
5. 6.96
6. 6
7. 5.97

```
List of 7
 $ : num 7.11
 $ : num 7.47
 $ : num 6.54
 $ : num 7.41
 $ : num 6.96
 $ : num 6
 $ : num 5.97
```

Now let's try to make this triple function a bit more generic, and instead call it multiply, and have it use an additional argument, factor: We can now choose with which factor we want to multiply our input. Calling triple is now the same as calling multiply with the factor argument equal to three.

In [22]:
```
oil_prices <- list(2.37, 2.49, 2.18, 2.47, 2.32, 2.00, 1.99)

multiply <- function(x, factor){
    factor * x
}

result <- lapply(oil_prices, multiply, factor=3)

result

str(result)
```

1. 7.11
2. 7.47
3. 6.54
4. 7.41
5. 6.96
6. 6
7. 5.97

```
List of 7
 $ : num 7.11
 $ : num 7.47
 $ : num 6.54
 $ : num 7.41
 $ : num 6.96
 $ : num 6
 $ : num 5.97
```

In [34]:
```
?lapply
```

To put it generally, lapply takes a vector or list X, and applies the function FUN to each of its members. If FUN requires additional arguments, you pass them after you've specified X and FUN (...). The output of lapply() is a list, the same length as X, where each element is the result of applying FUN on the corresponding element of X.

In [23]:
```
# The vector pioneers has already been created for you
pioneers <- c("GAUSS:1777", "BAYES:1702", "PASCAL:1623", "PEARSON:1857")
pioneers

# Split names from birth year
split_math <- strsplit(pioneers, split = ":")

split_math
class(split_math)


# Convert to lowercase strings: split_low
split_low <- lapply(split_math, tolower)



# Take a look at the structure of split_low
str(split_low)
```

'GAUSS:1777' 'BAYES:1702' 'PASCAL:1623' 'PEARSON:1857'

1. 'GAUSS' '1777'
2. 'BAYES' '1702'
3. 'PASCAL' '1623'
4. 'PEARSON' '1857'

'list'

```
List of 4
 $ : chr [1:2] "gauss" "1777"
 $ : chr [1:2] "bayes" "1702"
 $ : chr [1:2] "pascal" "1623"
 $ : chr [1:2] "pearson" "1857"
```

In [24]:
```
# Code from previous exercise:
pioneers <- c("GAUSS:1777", "BAYES:1702", "PASCAL:1623", "PEARSON:1857")
split <- strsplit(pioneers, split = ":")
split_low <- lapply(split, tolower)

# Write function select_first()
select_first <- function(x) {
  x[1]
}

# Apply select_first() over split_low: names
names <- lapply(split_low, select_first)

names

# Write function select_second()
select_second <- function(x) {
  x[2]
}



# Apply select_second() over split_low: years
years <- lapply(split_low, select_second)

years
```

1. 'gauss'
2. 'bayes'
3. 'pascal'
4. 'pearson'

1. '1777'
2. '1702'
3. '1623'
4. '1857'

2.6.1.1 anonymous functions

Previously, you learned that functions in R are objects in their own right. This means that they aren't automatically bound to a name. When you create a function, you can use the assignment operator to give the function a name. It's perfectly possible, however, to not give the function a name. This is called an **anonymous function**:

In [41]:
```
# Named function
triple <- function(x) { 3 * x }

# Anonymous function with same implementation
function(x) { 3 * x }

# Use anonymous function inside lapply()
lapply(list(1,2,3), function(x) { 3 * x })
```

```
function (x)
{
    3 * x
}
```

1. 3
2. 6
3. 9

2.6.1.2 Apply functions that return NULL

In all of the previous exercises, it was assumed that the functions that were applied over vectors and lists actually returned a meaningful result. For example, the tolower() function simply returns the strings with the characters in lowercase. This won't always be the case. Suppose you want to display the structure of every element of a list. You could use the str() function for this, which returns NULL:

In [42]:
```
lapply(list(1, "a", TRUE), str)
```

```
 num 1
 chr "a"
 logi TRUE
```

1. NULL
2. NULL
3. NULL

This call actually returns a list, the same size as the input list, containing all NULL values. On the other hand calling

2.6.2 sapply

Before, I talked about how lapply can be used to apply a function over each and every element of a list or a vector. The key thing of the lapply function is that its output is always a list. That's because an R function can return any R object. Also, the class of the R object it returns can differ depending on the input. When lapply is used to apply such a function over all elements in an input

list or vector, it needs a list to store these results, because a list is able to contain heterogeneous content. However, you can think of many cases where the function always returns the same type of object over and over.

In [44]:
```
cities <- c("New York", "Paris", "london", "Tokyo", "Rio de Janeiro", "Talca")

lapply(cities, nchar)
```

1. 8
2. 5
3. 6
4. 5
5. 14
6. 5

We had a vector, cities. We used lapply with the nchar function. As a result, we obtained a list with the length of each of the cities names. But wait! These values could very well fit into a simple vector as well! Because all have the same type.

In [45]:
```
unlist(lapply(cities, nchar))
```

8 5 6 5 14 5

Even, we already tried to solve this by using the unlist function to convert a list to a vector as follows: But behold! there's an easier way to tackle the case in which all the results have the same type by using the **sapply function**.

In [46]:
```
sapply(cities, nchar)
```

New York
8
Paris
5
london
6
Tokyo
5
Rio de Janeiro
14
Talca
5

In [48]:
```
sapply(cities, nchar, USE.NAMES = FALSE)
```

```
8 5 6 5 14 5
```

Now, what would happen if the function you want to apply over the input, each time returns a vector containing two values instead of one?

In [2]:
```
cities <- c("New York", "Paris", "london", "Tokyo", "Rio de Janeiro", "Talca")


first_and_last <- function(name) {
    name <- gsub(" ", "", name)
    letters <- strsplit(name, split = "")[[1]]
    c(first = min(letters), last = max(letters))

}

sapply(cities, first_and_last)
```

A matrix: 2 × 6 of type chr

	New York	Paris	london	Tokyo	Rio de Janeiro	Talca
first	e	a	d	k	a	a
last	Y	s	o	y	R	T

2.6.3 vapply

vapply is quite similar to sapply. Under the hood, it uses lapply and then tries to simplify the result. However, when using vapply, you have to **explicitly say what the type of the return value will be**. In sapply, this is not required nor possible.

2.6.3.1 sapply() vs vapply()

Let's take a look at a case where sapply and vapply act quite similarly,

In [5]:
```
cities <- c("New York", "Paris", "london", "Tokyo", "Rio de Janeiro", "Talca")

sapply(cities, nchar)

vapply(cities, nchar)
```

New York
8
Paris
5
london
6
Tokyo
5
Rio de Janeiro
14
Talca
5

```
Error in vapply(cities, nchar): argument "FUN.VALUE" is missing, with no defaul
t
Traceback:

1. vapply(cities, nchar)
```

What happen?

In [9]:
```
?vapply

#vapply(X, FUN, FUN.VALUE, ..., USE.NAMES = TRUE)
```

In [10]:
```
vapply(cities, nchar, numeric(1))
```

New York
8
Paris
5
london
6
Tokyo
5
Rio de Janeiro
14
Talca
5

nchar is a function that returns a single number, which is a numeric vector of length 1. We can template this output using the **numeric()** function, by setting **FUN.VALUE** to **numeric(1)**, which tells the **vapply function that nchar() should return a single numerical value**. The result is exactly the same as the sapply function from before. However, this 'pre-specification' of FUN's return value makes vapply a safer alternative to sapply.

To understand this, let's re-use another example from our discussion of sapply, where we extracted the first and last letters of the cities' names.

In [12]:
```
cities <- c("New York", "Paris", "london", "Tokyo", "Rio de Janeiro", "Talca")


first_and_last <- function(name) {
    name <- gsub(" ", "", name)
    letters <- strsplit(name, split = "")[[1]]
    c(first = min(letters), last = max(letters))

}

sapply(cities, first_and_last)

vapply(cities, first_and_last, character(2))
```

A matrix: 2 × 6 of type chr

	New York	Paris	london	Tokyo	Rio de Janeiro	Talca
first	e	a	d	k	a	a
last	Y	s	o	y	R	T

A matrix: 2 × 6 of type chr

	New York	Paris	london	Tokyo	Rio de Janeiro	Talca
first	e	a	d	k	a	a
last	Y	s	o	y	R	T

2.7 Some useful functions in R

You surely have some idea of the huge amount of useful functions that are available in R. The functions of the apply family that you've learned about before are just an illustration of this diversity.

2.7.1 Mathematical functions

In [37]:
```
a1 <- c(2.4, -8.1, 1.3, -3.4)
a2 <- c(-7.4, 2.6, 6.9, -5.0)

mean(c(sum(round(abs(a1))), sum(round(abs(a2)))))
```

18

2.7.2 Data structure functions

In [41]:
```
li <- list(log = TRUE,
           ch = "hello",
           int_vec = sort(rep(seq(8, 2, by= -2), times = 2)))

li
```

$log
TRUE
$ch
'hello'
$int_vec
2 2 4 4 6 6 8 8

In [42]:
```
a <- seq(8, 2, by = -2)

a
```

8 6 4 2

In [45]:
```
b <- rep(c(8, 6, 4, 2), times = 2)

b
```

8 6 4 2 8 6 4 2

In [46]:
```
c <- sort(c(8, 6, 4, 2, 8, 6, 4, 2))

c
```

2 2 4 4 6 6 8 8

2.7.2.1 str()

In [47]:
```
str(li)
```

```
List of 3
 $ log    : logi TRUE
 $ ch     : chr "hello"
 $ int_vec: num [1:8] 2 2 4 4 6 6 8 8
```

2.7.2.2 is.*( ) and as.*( )

In [48]:
```
is.list(li)
```

TRUE

In [49]:
```
is.list(c(8, 6, 4, 2))
```

FALSE

In [54]:
```
li2 <- c(8, 6, 4, 2)

is.list(li2)

li2 <- as.list(c(8, 6, 4, 2))

is.list(li2)
```

FALSE

TRUE

2.7.2.3 append( ) and rev( )

In [56]:
```
str(li)
str(append(li, rev(li)))
```

```
List of 3
 $ log    : logi TRUE
 $ ch     : chr "hello"
 $ int_vec: num [1:8] 2 2 4 4 6 6 8 8
List of 6
 $ log    : logi TRUE
 $ ch     : chr "hello"
 $ int_vec: num [1:8] 2 2 4 4 6 6 8 8
 $ int_vec: num [1:8] 2 2 4 4 6 6 8 8
 $ ch     : chr "hello"
 $ log    : logi TRUE
```

In [ ]:
```

```